IACD Projeto

Project Description

Enquanto estudantes, os autocarros SMTUC são um meio de transporte não só conveniente como também necessário para muitos, porém, alguns dos seus utilizadores podem sentir que algumas zonas de Coimbra têm muito mais cobertura que outras. Alguma dessa cobertura pode ser necessária mas poderia eventualmente ser substituída por paragens noutros locais que poderão ser mais necessários e daí surge o nosso projeto.

Este projeto tem o objetivo de, através do *crowdsourcing*, conseguirmos a criação de mais paragens através do acesso simulado à aplicação de requisição de autocarros (ex: MOOVIT) por vários utilizadores.

Foi utilizado um dataset em formato .txt que consiste em todas as paragens dos autocarros SMTUC até ao dia da recolha do dataset obtido no site oficial: https://dados.gov.pt/pt/datasets/paragens-autocarros/#/resources.

Arquitetura

1. Camada de Dados (Data Source)

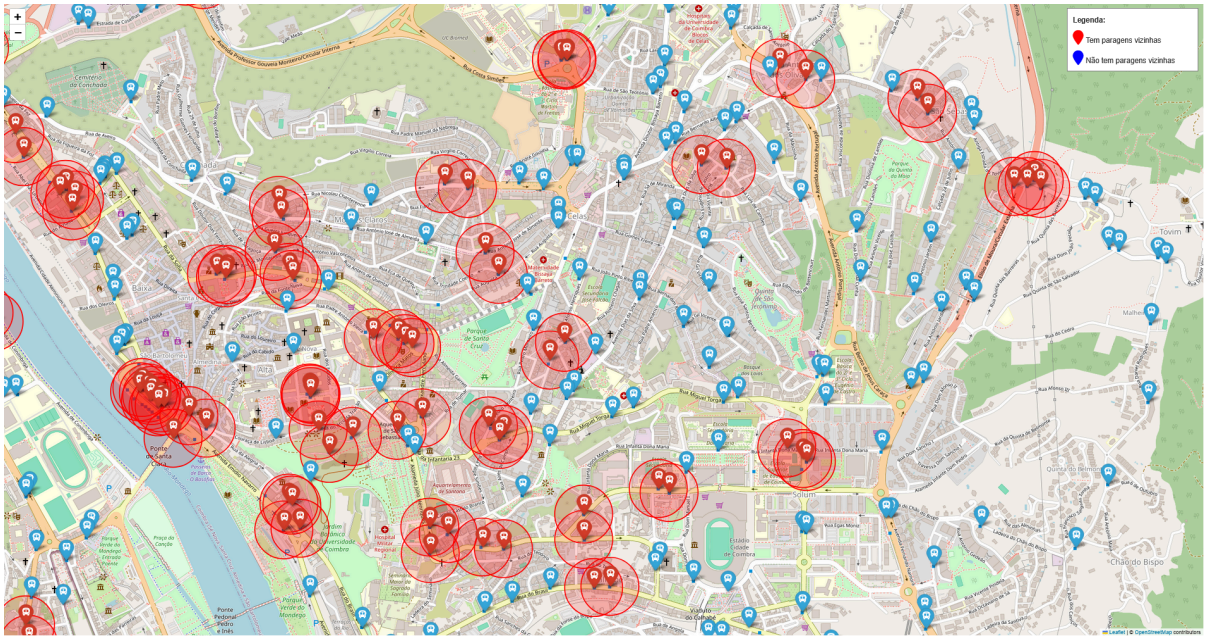
- **Estático (Infraestrutura):** Ficheiro `data/stops_smtuc.txt`.
  - **Dados:** Ficheiro txt que contém a localização fixa das paragens.
- **Dinâmico (Simulação):** Script `src/producer/producer.py`.
  - **Dados:** JSON com a mensagem `{event_type, timestamp, user_lat`, `user_lon`, `zone_hint}`.
  - **Função:** Simula utilizadores a abrir a App SMTUC em zonas sem cobertura ("Crowd Sourcing") sendo enviados dados para o Kafka a cada 0.5s.
  - **Lógica:** Gera "clusters" de pedidos em zonas específicas (ex: Planalto Santa Clara).

- **Exemplo:**

```
👉 Pedido em: Hospital dos Covões
👉 Pedido em: Estação Coimbra-A (Rio)
👉 Pedido em: Monte Formoso
👉 Pedido em: Retail Park Eiras
👉 Pedido em: HUC - Hospitais Univ.
👉 Pedido em: Zona ESEC
👉 Pedido em: Solum - Alma Shopping
👉 Pedido em: Planalto Sta Clara
👉 Pedido em: Estação Coimbra-B
👉 Pedido em: Estação Coimbra-A (Rio)
👉 Pedido em: IPC - Escola Agrária (Bencanta)
👉 Pedido em: Estação Coimbra-B
👉 Pedido em: Estação Coimbra-B
👉 Pedido em: Solum - Alma Shopping
👉 Pedido em: Forum Coimbra
👉 Pedido em: Zona ESEC
👉 Pedido em: HUC - Hospitais Univ.
👉 Pedido em: Monte Formoso
👉 Pedido em: Pólo II - Engenharia
```

2. Camada de Visualização (Presentation Layer)

- **Análise de Redundância:**
  - Script: `visualizar_mapa.py` (Pandas + Folium).
  - Output: `mapa_smtuc.html`.
    - Mostra visualmente onde existem paragens sobrepostas (<100m) marcadas a vermelho e a azul as paragens que não têm paragens sobrepostas.
    - *No caso de haverem 2 paragens muito perto com o mesmo exato ID, são consideradas como paragens de ida e volta



- **Análise de Cobertura:**
  - Output: Tabela na Consola do Spark. Lista as coordenadas exatas onde é necessário enviar equipas técnicas para instalar novas paragens.
- **Confirmação de Hipóteses**

- O mapa criado serve também para nós podermos visualizar se a paragem que está a ser proposta tem sentido ou está a ser marcada de forma errada

3. Camada de Ingestão (Consumer Layer)

- **Tecnologia:**
  - Apache Kafka (`wurstmeister/kafka`)
  - Zookeeper (`confluentinc/cp-zookeeper:7.3.0`).
- **Configuração:**
  - **Porta Externa (9092):** Recebe dados do Python no Windows.
  - **Porta Interna (29092):** Envia dados para o Spark dentro do Docker.
- **Tópico:** `smtuc_demand` atua como um buffer que recebe os dados do Producer e distribui para dois consumidores:
  - **Apache Spark:** Para processamento, agregação e persistência em base de dados.
  - **Consumer Monitor:** Script de monitorização (`consumer_monitor.py`) para validação de logs em tempo real.
  - **Exemplo**:

```
RAW INGEST | Pólo II - Engenharia (40.1864, -8.4164) às 2025-12-11 22:39:42
RAW INGEST | Estação Coimbra-B (40.2243, -8.4412) às 2025-12-11 22:39:43
RAW INGEST | Estação Coimbra-A (Rio) (40.2084, -8.4312) às 2025-12-11 22:39:43
RAW INGEST | Solum - Alma Shopping (40.2014, -8.4116) às 2025-12-11 22:39:44
RAW INGEST | Retail Park Eiras (40.2332, -8.4241) às 2025-12-11 22:39:44
RAW INGEST | Planalto Sta Clara (40.1953, -8.4450) às 2025-12-11 22:39:45
RAW INGEST | Planalto Sta Clara (40.1945, -8.4455) às 2025-12-11 22:39:45
RAW INGEST | Monte Formoso (40.2200, -8.4323) às 2025-12-11 22:39:46
RAW INGEST | Zona ESEC (40.2088, -8.4200) às 2025-12-11 22:39:46
RAW INGEST | Planalto Sta Clara (40.1948, -8.4449) às 2025-12-11 22:39:47
RAW INGEST | Zona ESEC (40.2086, -8.4199) às 2025-12-11 22:39:47
RAW INGEST | Loja Cidadão / Baixa (40.2102, -8.4281) às 2025-12-11 22:39:48
RAW INGEST | Quinta da Portela (40.1908, -8.4051) às 2025-12-11 22:39:48
RAW INGEST | Hospital dos Covões (40.1971, -8.4368) às 2025-12-11 22:39:49
RAW INGEST | Estação Coimbra-A (Rio) (40.2086, -8.4308) às 2025-12-11 22:39:49
RAW INGEST | Pólo I - Alta Universitária (40.2078, -8.4264) às 2025-12-11 22:39:50
RAW INGEST | HUC - Hospitais Univ. (40.2117, -8.4167) às 2025-12-11 22:39:50
RAW INGEST | Monte Formoso (40.2201, -8.4316) às 2025-12-11 22:39:51
RAW INGEST | Quinta da Portela (40.1908, -8.4054) às 2025-12-11 22:39:51
```

4. Camada de Processamento (Processing Layer)

- **Tecnologia:** Apache Spark Streaming (`apache/spark:latest` - Scala 2.13).
- **Script:** `src/main.py` .
- **Lógica:**
  - Utilização de **Broadcast Variables** para carregar as paragens estáticas e **UDFs** (User Defined Functions) para calcular a distância Haversine de cada pedido à paragem mais próxima em tempo real.
  - **Leitura:** Structured Streaming ligado ao Kafka.
  - **Transformação:** `Grid Aggregation` . Arredonda coordenadas GPS (3 casas decimais) para criar "zonas de 100m".

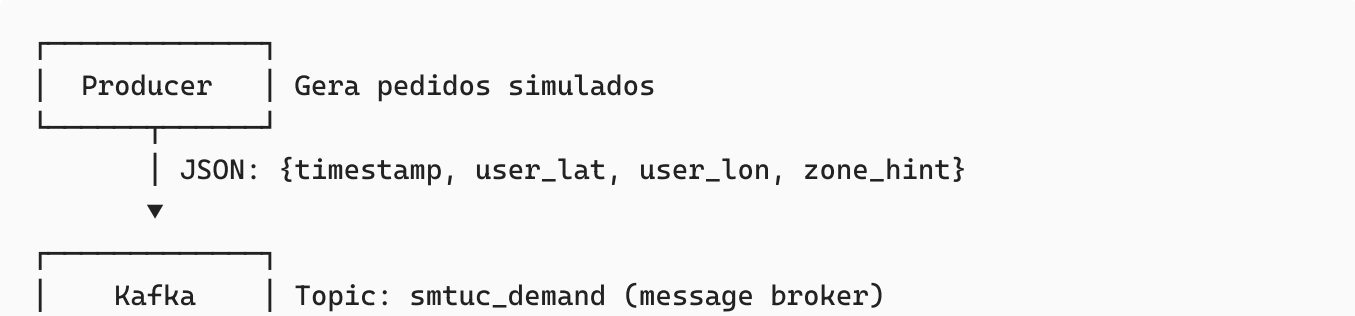
- **Trigger de Alerta:** O sistema identifica zonas críticas onde a lacuna de cobertura (`percentage_gap`) é > 50% e existem pelo menos 5 pedidos.
- **Escrita (Output):** O Spark separa o fluxo em dois *streams*:
  1. **Raw Stream:** Guarda todos os pedidos individuais.
  2. **Aggregated Stream:** Calcula métricas por zona (total de pedidos, % de pedidos sem paragem próxima, distância média) a cada 30 segundos.

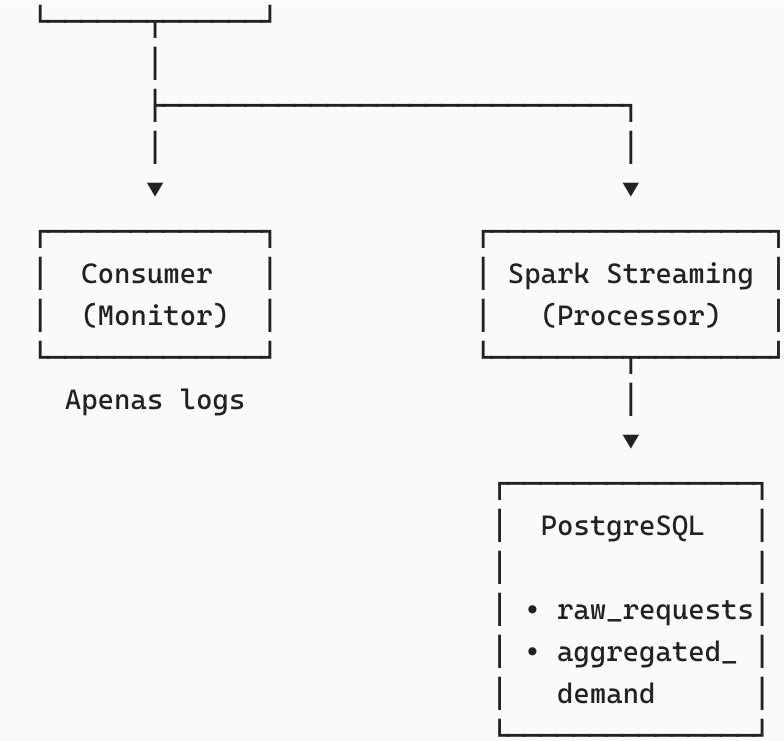
5. Camada de Armazenamento Persistente (Storage Layer)

- **Tecnologia: PostgreSQL**.
- **Componente de Ligação:** O próprio Apache Spark (`main.py`) via conector JDBC (`postgresql-42.7.1.jar`).
  - Este script funciona em paralelo ao Spark, escutando o mesmo tópico.
  - A sua função é garantir que nenhum dado é perdido, persistindo cada requisição de paragem (`request_stop`) numa tabela relacional estruturada.
    - No caso de ser requisitada uma paragem numa zona que já existe a menos de 100 metros, a coluna *boolean* "extra_stop_needed" imprime 0, caso contrário imprimirá 1.
- **Estrutura de Dados:**
  - Base de dados com duas tabelas:
    - **`raw_requests` :** Registo histórico de cada "ping" de utilizador com classificação de necessidade (`extra_stop_needed`).
    - **`aggregated_demand` :** Tabela analítica com o resumo da procura por zona geográfica, pronta para visualização.

zona_sugerida	total_requests	requests_needing_stop	percentage_gap	dist_avg
Estação Coimbra-B	399	399	100	180.69
Planalto Sta Clara	363	363	100	187.01
Retail Park Eiras	347	347	100	162.10
Retail Park Eiras	338	338	100	203.74
Planalto Sta Clara (Gap)	284	284	100	189.70
Forum Coimbra	256	256	100	280.61
Forum Coimbra	251	251	100	282.60
Estação Coimbra-B	190	190	100	194.62
Loja Cidadão / Baixa	143	143	100	159.09
Loja Cidadão / Baixa	137	137	100	140.94
Forum Coimbra	58	58	100	238.42
Forum Coimbra	53	53	100	242.40
Estação Coimbra-B	5	5	100	219.77

Desenho da Arquitetura





Tecnologias

- **Kubernetes**:
  - Utilizado para criação de contentores, correndo o Kafka, Zookeper e Spark em Pods isolados
- **Apache Kafka (*Broker* de Mensagens):**
  - Atua como a camada de ingestão de alta velocidade e baixa latência. Foi escolhido para desacoplar a produção de dados (os milhares de utilizadores na App) do processamento. O Kafka garante que, mesmo que o Spark esteja ocupado ou lento, nenhum pedido de "nova paragem" se perde, ficando em fila (buffer) no tópico `smtuc_demand`.
- **Apache Spark Structured Streaming (Stream Processing):**
  - Apesar do número reduzido de dados e por não se tratar de Big Data, não seria expectável o uso deste mecanismo se fosse um projeto real, porém para fins académicos, e pelo natural *scope* da cadeira, utilizámo-lo como processamento de dados a tempo real
- **Pandas & Folium (Carregamento & Visualização de Dados):**
  - **Pandas:** Utilizado para carregar e manipular o dataset estático (`stops.txt`) e identificar redundâncias iniciais.
  - **Folium:** Biblioteca utilizada para a renderização geoespacial, permitindo criar um mapa interativo HTML que serve como *Proof of Concept* visual para a administração dos SMTUC validarem as zonas de intervenção.
- **PostgreSQL**

- Inicialmente, foi utilizado o SQLite por ser não só a mais direta como a mais leve para utilizar em projetos académicos, porém enfrentámos um problema no qual essa Base de Dados guarda os dados **localmente** o que não é expectável para o nosso projeto. Como tal, e porque queremos persistência real, alterámos para o uso de PostgreSQL, permitindo também um streaming otimizado.

Limitações e Trabalho Futuro

As limitações deste trabalho começam com o dataset; Em análise inicial tinhamos a ideia de que cada linha enumerava uma viagem e não apenas uma paragem, pelo que dias antes do Project Proposal foi alterado o Scope do projeto para poder acomodar tais diferenças.

Já mencionadas brevemente na parte das tecnologias, tivemos algumas limitações no uso de algumas das ferramentas devido a escolhas erradas ao iniciar o projeto, principalmente no uso de SQLite. Outra limitação, também ligada a SQLite foi a utilização de Spark com o mesmo, que nos deu alguns problemas.

Como trabalho futuro, sabemos que atualmente o nosso projeto está condicionado a não considerar IDs de pessoas e está sujeito a manipulação de dados por um só utilizador pelo que, para o futuro, o necessário a fazer logo de início é começar por introduzir um mecanismo no qual apenas é considerada uma sugestão de um utilizador num raio de 200m (por exemplo). Este número é escolhido para que não haja abuso do sistema e ainda assim permitir ao utilizador dar informação valiosa ao servidor.

Sendo um pouco mais ambicioso, e caso a sugestão anterior fosse conseguida, seria ótimo a integração com uma das aplicações já existentes (Moovit por exemplo).

Nomes e Números de Estudante Respetivos

- Afonso Rodrigues - 2020242400
- Constança Correia - 2020242519